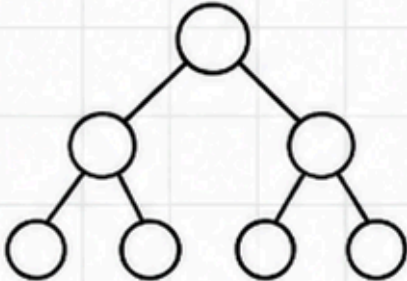
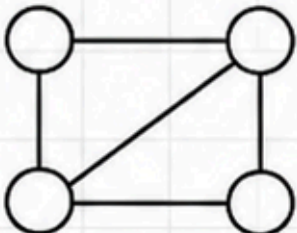


# Data Structures

## Definition

A data structure is a way of organizing and storing data so that it can be used efficiently. Algorithms are step-by-step procedures or formulas for solving problems using data structures.

## Basic Data Structures (with Drawing)

| <u>Structure</u> | <u>Drawing</u>                                                                       | <u>Use</u>                                                       |
|------------------|--------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Array            |  | Stores elements of the same type in contiguous memory locations. |
| Linked List      |  | Stores elements in nodes; each node points to the next.          |
| Stack            |   | Follows LIFO (Last In, First Out) principle.                     |
| Queue            |  | Follows FIFO (First In, First Out) principle.                    |
| Tree             |  | Hierarchical structure with a root and subtrees.                 |
| Graph            |   | Collection of nodes (or vertices) connected by edges.            |



# Algorithms

## Definition

An algorithm is a finite set of well-defined instructions or steps to solve a problem or accomplish a task. It must produce output for given input and terminate after a finite number of steps.

## Characteristics of an Algorithm

1. Input : Takes zero or more inputs.
2. Output : Produces at least one output.
3. Definiteness : Each step is clear and unambiguous.
4. Finiteness : Terminates after a finite number of steps.
5. Effectiveness : Each step is basic and executable in finite time.

## Types of Algorithms

1. Recursive Algorithm  
An algorithm that solves a problem by calling itself with smaller instances of the same problem.
2. Iterative Algorithm  
An algorithm that solves a problem by repeating a set of steps until a condition is met.
3. Divide and Conquer Algorithm  
An algorithm that divides a problem into smaller subproblems, solves them recursively, and combines their solutions.

## Importance of Algorithms

1. Provides a step-by-step solution to a problem.
2. Helps in writing efficient and error-free programs.
3. Improves code readability and maintainability.
4. Essential in analyzing time and space complexity.

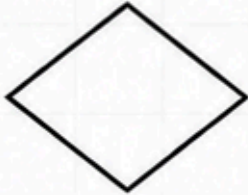


# Algorithm Representation

## Flowchart

A flowchart is a graphical representation of an algorithm using standard symbols connected by arrows.

## Basic Symbols in a Flowchart

| <u>Symbol</u>                                                                       | <u>Name</u>        | <u>Use</u>                                     |
|-------------------------------------------------------------------------------------|--------------------|------------------------------------------------|
|    | Terminator         | Indicates start or end of the algorithm.       |
|  | Input / Output     | Represents input or output.                    |
|  | Process            | Represents an instruction or step.             |
|  | Decision           | Represents a decision point with Yes/No paths. |
|  | Flow Line          | Shows the direction of flow.                   |
|  | Connector          | Connects flow lines on the same page.          |
|  | Off-page Connector | Connects flow lines to another page.           |

## Uses of Flowcharts

1. Helps in planning and understanding a solution.
2. Improves communication.
3. Helps in debugging and tracing errors.
4. Serves as a reference for future use.



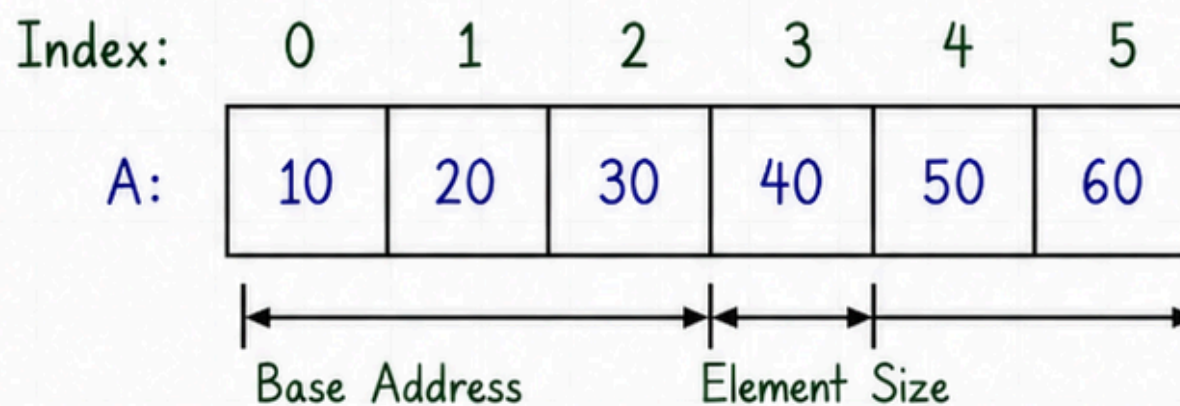
# Arrays

## Definition

An array is a collection of elements of the same type stored at contiguous memory locations. Each element can be accessed using an index.

## Representation

Array: A



## Characteristics

1. Fixed size: The size of an array is fixed once declared.
2. Homogeneous: All elements must be of the same type.
3. Contiguous memory: Elements are stored in adjacent memory locations.
4. Random access: Any element can be accessed directly using its index.

## Operations

1. Traversal: Visit each element of the array.
2. Insertion: Add an element at a specified position.
3. Deletion: Remove an element from a specified position.
4. Searching: Find the position of a given element.

## Applications

1. Storing lists of data.
2. Implementing other data structures like stacks, queues, etc.
3. Efficient data access in various algorithms.

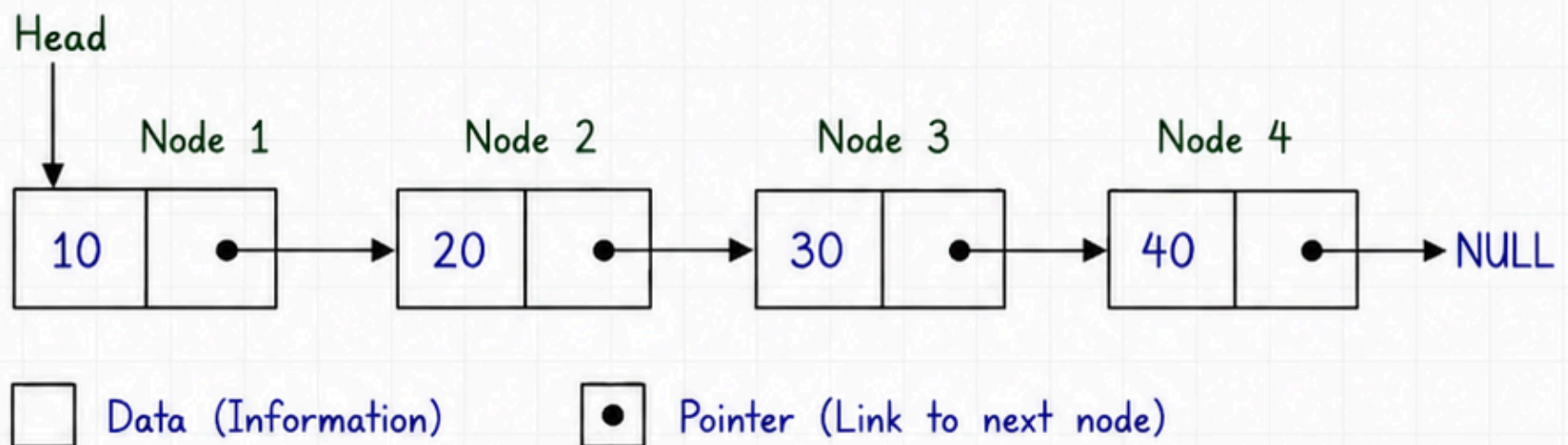


# Linked Lists

## Definition

A linked list is a linear data structure where elements, called nodes, are not stored at contiguous memory locations. Each node contains data and a reference (link) to the next node in the sequence.

## Singly Linked List Representation



## Characteristics

1. Dynamic size: A linked list can grow or shrink during program execution.
2. Non-contiguous memory: Nodes are stored at non-adjacent memory locations.
3. Efficient insertion and deletion: Insertion or deletion of nodes is efficient compared to arrays.
4. Sequential access: Elements must be accessed sequentially starting from the head.
5. Extra memory: Additional memory is required to store pointers.

## Basic Operations

1. Traversal: Visit each node in the list.
2. Insertion: Add a new node at the beginning, end, or a specific position.
3. Deletion: Remove a node from the beginning, end, or a specific position.
4. Search: Find a node with a given value.

## Applications

1. Implementing other data structures like stacks, queues, and graphs.
2. Dynamic memory allocation.
3. Polynomial representation.

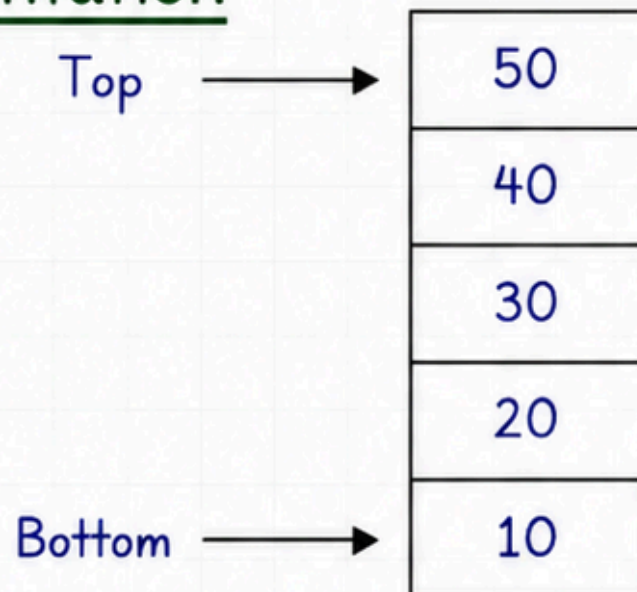


# Stacks

## Definition

A stack is a linear data structure that follows the LIFO (Last In, First Out) principle. Elements are added and removed from the same end, called the top.

## Representation



Top points to the topmost element.

New elements are added or removed from the top.

## Characteristics

1. Follows LIFO (Last In, First Out) principle.
2. Insertion and deletion occur only at the top.
3. Dynamic size: A stack can grow or shrink during execution.
4. Efficient operations: Push and pop operations are efficient.

## Basic Operations

1. Push: Add an element to the top of the stack.
2. Pop: Remove the top element from the stack.
3. Peek/Top: Get the value of the top element without removing it.
4. IsEmpty: Check whether the stack is empty.
5. IsFull: Check whether the stack is full (for fixed-size stacks).

## Applications

1. Function call management (call stack).
2. Expression evaluation and conversion.
3. Parentheses matching.
4. Backtracking (e.g., maze, N-queens problem).

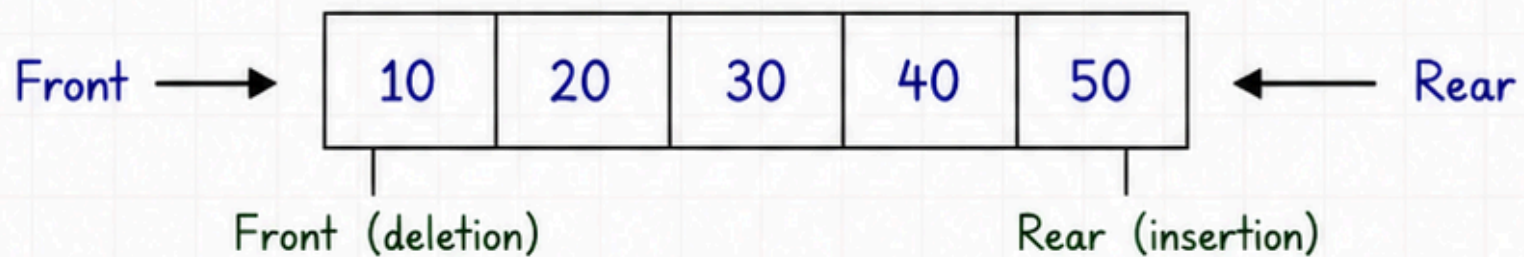


# Queues

## Definition

A queue is a linear data structure that follows the FIFO (First In, First Out) principle. Elements are added at the rear and removed from the front.

## Representation



Elements are inserted at the rear and deleted from the front.

## Characteristics

1. Follows FIFO (First In, First Out) principle.
2. Insertion occurs only at the rear.
3. Deletion occurs only at the front.
4. Dynamic size: A queue can grow or shrink during execution.
5. Efficient operations: Enqueue and dequeue are efficient.

## Basic Operations

1. Enqueue: Add an element at the rear of the queue.
2. Dequeue: Remove the element from the front of the queue.
3. Front/Ppeek: Get the value of the front element without removing it.
4. IsEmpty: Check whether the queue is empty.
5. IsFull: Check whether the queue is full (for fixed-size queues).

## Applications

1. CPU scheduling and task management.
2. Handling requests in web servers.
3. Printer spooling.
4. Breadth-First Search (BFS) in graphs.

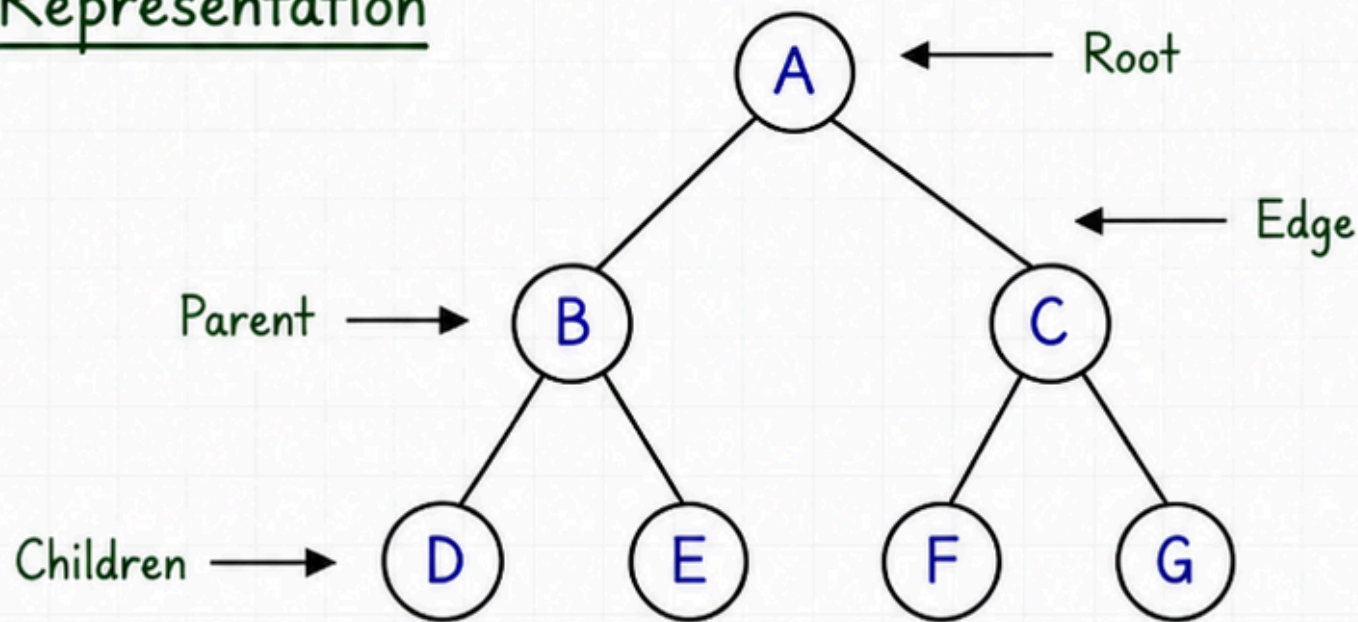


# Trees

## Definition

A tree is a non-linear data structure consisting of nodes connected by edges. It has a hierarchical structure with a single root node and zero or more subtrees.

## Representation



## Characteristics

1. A tree has exactly one root node.
2. Each node (except the root) has exactly one parent.
3. A node can have zero or more children.
4. There are no cycles in a tree.
5. A tree with  $n$  nodes has exactly  $n - 1$  edges.

## Basic Operations

1. Traversal: Visit all nodes in a specific order.
2. Insertion: Add a new node to the tree.
3. Deletion: Remove a node from the tree.
4. Searching: Find a node with a given value.
5. Finding Parent/Child: Identify parent or children of a node.

## Applications

1. Representing hierarchical data (e.g., file systems, organization charts).
2. Expression trees in compilers.
3. Database indexing (e.g., B-trees).
4. Decision making (e.g., game trees, AI).



# Graphs

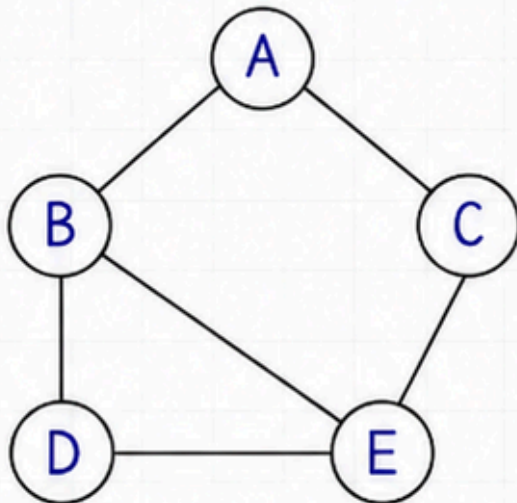
## Definition

A graph is a non-linear data structure consisting of vertices (nodes) and edges. It can be directed or undirected.

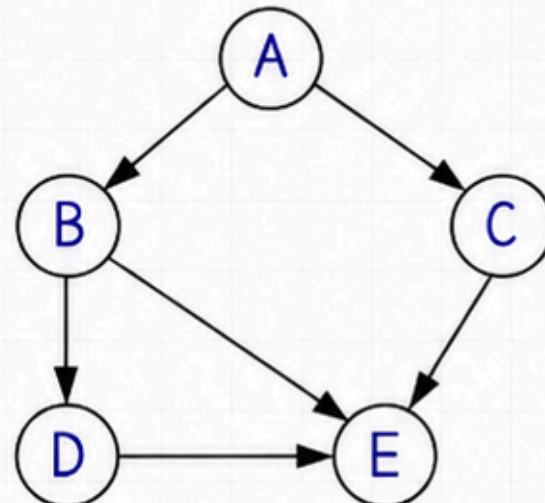
Graphs can be used to represent networks and relationships.

## Representation

Undirected Graph



Directed Graph



## Characteristics

1. A graph consists of a set of vertices ( $V$ ) and a set of edges ( $E$ ).
2. Edges can be undirected (no direction) or directed (one direction).
3. A simple undirected graph has no self-loops and no parallel edges.
4. A graph can be connected or disconnected.
5. A weighted graph has edges with associated weights or costs.

## Basic Operations

1. Add Vertex: Add a new vertex to the graph.
2. Add Edge: Add an edge between two vertices.
3. Remove Vertex: Delete a vertex and all its incident edges.
4. Remove Edge: Delete an edge between two vertices.
5. Check Adjacency: Determine if two vertices are adjacent.

## Applications

1. Social networks (friends, followers, connections).
2. Road maps and navigation systems.
3. Web link analysis and ranking (e.g., PageRank).
4. Network design and communication systems.



Today's  
vibe...



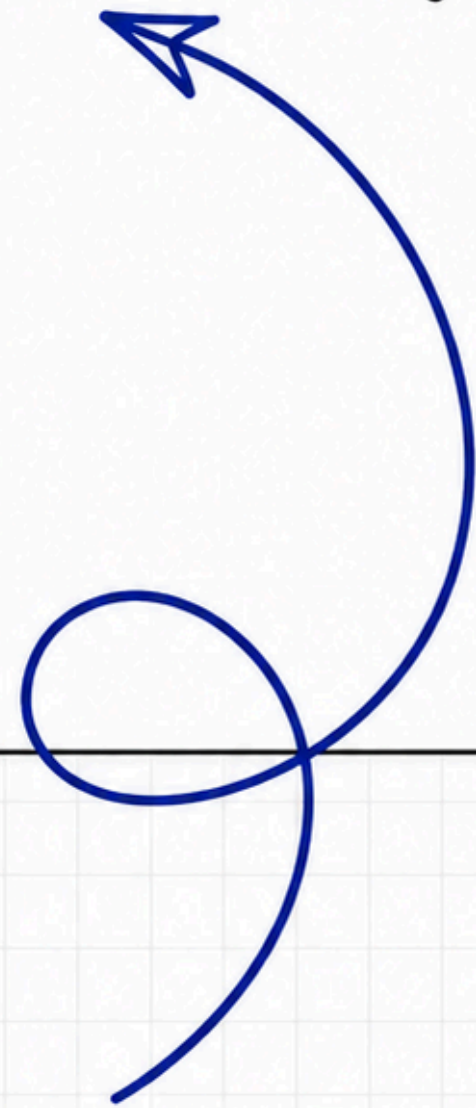
Akash • Python & Tech Enthusiast  


1,584 posts      681K followers      18 following

Education

-  | Learn Python, ML, and AI Fast
-  | Work: [contact@legitpython.com](mailto:contact@legitpython.com)
-  | Get Python Notes + Projects
-  [bit.ly/pycodehubb](https://bit.ly/pycodehubb) and 3 more

 Pycode Updates       Add



Follow **@Pycode.hubb** for more such  
coding, Python, AI, and Data Science  
content 